



Choupo

The Open Source Chemical Process Simulator

User Guide

Version v0.2.0

18 June 2026

Copyright © 2026 Vítor Geraldês and Pedro Mendes.

Authors: Vítor Geraldês and Pedro Mendes.

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.

Code examples, Choupo case files, and other machine-readable examples are licensed under GPL-3.0-or-later.

Typeset in L^AT_EX.

Acknowledgment. Choupo was conceived and architected by Vítor Geraldês. Substantial parts of the initial implementation, documentation, and tutorial corpus were produced with assistance from Anthropic Claude Code using Claude Opus/Fable models. This guide is published under human curation, review, and editorial responsibility by Vítor Geraldês and Pedro Mendes.

License

Attribution-ShareAlike 4.0 International

=====
Creative Commons Corporation ("Creative Commons") is not a law firm and does not provide legal services or legal advice. Distribution of Creative Commons public licenses does not create a lawyer-client or other relationship. Creative Commons makes its licenses and related information available on an "as-is" basis. Creative Commons gives no warranties regarding its licenses, any material licensed under their terms and conditions, or any related information. Creative Commons disclaims all liability for damages resulting from their use to the fullest extent possible.

Using Creative Commons Public Licenses

Creative Commons public licenses provide a standard set of terms and conditions that creators and other rights holders may use to share original works of authorship and other material subject to copyright and certain other rights specified in the public license below. The following considerations are for informational purposes only, are not exhaustive, and do not form part of our licenses.

Considerations for licensors: Our public licenses are intended for use by those authorized to give the public permission to use material in ways otherwise restricted by copyright and certain other rights. Our licenses are irrevocable. Licensors should read and understand the terms and conditions of the license they choose before applying it. Licensors should also secure all rights necessary before applying our licenses so that the public can reuse the material as expected. Licensors should clearly mark any material not subject to the license. This includes other CC-licensed material, or material used under an exception or limitation to copyright. More considerations for licensors: wiki.creativecommons.org/Considerations_for_licensors

Considerations for the public: By using one of our public licenses, a licensor grants the public permission to use the licensed material under specified terms and conditions. If the licensor's permission is not necessary for any reason--for example, because of any applicable exception or limitation to copyright--then that use is not regulated by the license. Our licenses grant only permissions under copyright and certain other rights that a licensor has authority to grant. Use of the licensed material may still be restricted for other reasons, including because others have copyright or other rights in the material. A licensor may make special requests, such as asking that all changes be marked or described. Although not required by our licenses, you are encouraged to respect those requests where reasonable. More considerations for the public: wiki.creativecommons.org/Considerations_for_licensees

=====
Creative Commons Attribution-ShareAlike 4.0 International Public License

By exercising the Licensed Rights (defined below), You accept and agree to be bound by the terms and conditions of this Creative Commons Attribution-ShareAlike 4.0 International Public License ("Public License"). To the extent this Public License may be interpreted as a contract, You are granted the Licensed Rights in consideration of Your acceptance of these terms and conditions, and the Licensor grants You such rights in consideration of benefits the Licensor receives from making the Licensed Material available under these terms and conditions.

Section 1 -- Definitions.

- a. Adapted Material means material subject to Copyright and Similar Rights that is derived from or based upon the Licensed Material and in which the Licensed Material is translated, altered, arranged, transformed, or otherwise modified in a manner requiring permission under the Copyright and Similar Rights held by the Licensor. For purposes of this Public License, where the Licensed Material is a musical work, performance, or sound recording, Adapted Material is always produced where the Licensed Material is synched in timed relation with a moving image.
- b. Adapter's License means the license You apply to Your Copyright and Similar Rights in Your contributions to Adapted Material in accordance with the terms and conditions of this Public License.
- c. BY-SA Compatible License means a license listed at creativecommons.org/compatiblelicenses, approved by Creative Commons as essentially the equivalent of this Public License.
- d. Copyright and Similar Rights means copyright and/or similar rights closely related to copyright including, without limitation, performance, broadcast, sound recording, and Sui Generis Database Rights, without regard to how the rights are labeled or categorized. For purposes of this Public License, the rights specified in Section 2(b)(1)-(2) are not Copyright and Similar

Rights.

- e. Effective Technological Measures means those measures that, in the absence of proper authority, may not be circumvented under laws fulfilling obligations under Article 11 of the WIPO Copyright Treaty adopted on December 20, 1996, and/or similar international agreements.
- f. Exceptions and Limitations means fair use, fair dealing, and/or any other exception or limitation to Copyright and Similar Rights that applies to Your use of the Licensed Material.
- g. License Elements means the license attributes listed in the name of a Creative Commons Public License. The License Elements of this Public License are Attribution and ShareAlike.
- h. Licensed Material means the artistic or literary work, database, or other material to which the Licensor applied this Public License.
- i. Licensed Rights means the rights granted to You subject to the terms and conditions of this Public License, which are limited to all Copyright and Similar Rights that apply to Your use of the Licensed Material and that the Licensor has authority to license.
- j. Licensor means the individual(s) or entity(ies) granting rights under this Public License.
- k. Share means to provide material to the public by any means or process that requires permission under the Licensed Rights, such as reproduction, public display, public performance, distribution, dissemination, communication, or importation, and to make material available to the public including in ways that members of the public may access the material from a place and at a time individually chosen by them.
- l. Sui Generis Database Rights means rights other than copyright resulting from Directive 96/9/EC of the European Parliament and of the Council of 11 March 1996 on the legal protection of databases, as amended and/or succeeded, as well as other essentially equivalent rights anywhere in the world.
- m. You means the individual or entity exercising the Licensed Rights under this Public License. Your has a corresponding meaning.

Section 2 -- Scope.

- a. License grant.
 - 1. Subject to the terms and conditions of this Public License, the Licensor hereby grants You a worldwide, royalty-free, non-sublicensable, non-exclusive, irrevocable license to exercise the Licensed Rights in the Licensed Material to:
 - a. reproduce and Share the Licensed Material, in whole or in part; and
 - b. produce, reproduce, and Share Adapted Material.
 - 2. Exceptions and Limitations. For the avoidance of doubt, where Exceptions and Limitations apply to Your use, this Public License does not apply, and You do not need to comply with its terms and conditions.
 - 3. Term. The term of this Public License is specified in Section 6(a).
 - 4. Media and formats; technical modifications allowed. The Licensor authorizes You to exercise the Licensed Rights in all media and formats whether now known or hereafter created, and to make technical modifications necessary to do so. The Licensor waives and/or agrees not to assert any right or authority to forbid You from making technical modifications necessary to exercise the Licensed Rights, including technical modifications necessary to circumvent Effective Technological Measures. For purposes of this Public License, simply making modifications authorized by this Section 2(a) (4) never produces Adapted Material.
 - 5. Downstream recipients.
 - a. Offer from the Licensor -- Licensed Material. Every recipient of the Licensed Material automatically receives an offer from the Licensor to exercise the Licensed Rights under the terms and conditions of this Public License.
 - b. Additional offer from the Licensor -- Adapted Material. Every recipient of Adapted Material from You automatically receives an offer from the Licensor to exercise the Licensed Rights in the Adapted Material under the conditions of the Adapter's License You apply.
 - c. No downstream restrictions. You may not offer or impose any additional or different terms or conditions on, or apply any Effective Technological Measures to, the Licensed Material if doing so restricts exercise of the Licensed Rights by any recipient of the Licensed Material.

6. No endorsement. Nothing in this Public License constitutes or may be construed as permission to assert or imply that You are, or that Your use of the Licensed Material is, connected with, or sponsored, endorsed, or granted official status by, the Licensor or others designated to receive attribution as provided in Section 3(a)(1)(A)(i).

b. Other rights.

1. Moral rights, such as the right of integrity, are not licensed under this Public License, nor are publicity, privacy, and/or other similar personality rights; however, to the extent possible, the Licensor waives and/or agrees not to assert any such rights held by the Licensor to the limited extent necessary to allow You to exercise the Licensed Rights, but not otherwise.
2. Patent and trademark rights are not licensed under this Public License.
3. To the extent possible, the Licensor waives any right to collect royalties from You for the exercise of the Licensed Rights, whether directly or through a collecting society under any voluntary or waivable statutory or compulsory licensing scheme. In all other cases the Licensor expressly reserves any right to collect such royalties.

Section 3 -- License Conditions.

Your exercise of the Licensed Rights is expressly made subject to the following conditions.

a. Attribution.

1. If You Share the Licensed Material (including in modified form), You must:
 - a. retain the following if it is supplied by the Licensor with the Licensed Material:
 - i. identification of the creator(s) of the Licensed Material and any others designated to receive attribution, in any reasonable manner requested by the Licensor (including by pseudonym if designated);
 - ii. a copyright notice;
 - iii. a notice that refers to this Public License;
 - iv. a notice that refers to the disclaimer of warranties;
 - v. a URI or hyperlink to the Licensed Material to the extent reasonably practicable;
 - b. indicate if You modified the Licensed Material and retain an indication of any previous modifications; and
 - c. indicate the Licensed Material is licensed under this Public License, and include the text of, or the URI or hyperlink to, this Public License.
2. You may satisfy the conditions in Section 3(a)(1) in any reasonable manner based on the medium, means, and context in which You Share the Licensed Material. For example, it may be reasonable to satisfy the conditions by providing a URI or hyperlink to a resource that includes the required information.
3. If requested by the Licensor, You must remove any of the information required by Section 3(a)(1)(A) to the extent reasonably practicable.

b. ShareAlike.

In addition to the conditions in Section 3(a), if You Share Adapted Material You produce, the following conditions also apply.

1. The Adapter's License You apply must be a Creative Commons license with the same License Elements, this version or later, or a BY-SA Compatible License.
2. You must include the text of, or the URI or hyperlink to, the Adapter's License You apply. You may satisfy this condition in any reasonable manner based on the medium, means, and context in which You Share Adapted Material.
3. You may not offer or impose any additional or different terms or conditions on, or apply any Effective Technological Measures to, Adapted Material that restrict exercise of the rights granted under the Adapter's License You apply.

Section 4 -- Sui Generis Database Rights.

Where the Licensed Rights include Sui Generis Database Rights that apply to Your use of the Licensed Material:

- a. for the avoidance of doubt, Section 2(a)(1) grants You the right to extract, reuse, reproduce, and Share all or a substantial portion of the contents of the database;
- b. if You include all or a substantial portion of the database contents in a database in which You have Sui Generis Database Rights, then the database in which You have Sui Generis Database Rights (but not its individual contents) is Adapted Material, including for purposes of Section 3(b); and
- c. You must comply with the conditions in Section 3(a) if You Share all or a substantial portion of the contents of the database.

For the avoidance of doubt, this Section 4 supplements and does not replace Your obligations under this Public License where the Licensed Rights include other Copyright and Similar Rights.

Section 5 -- Disclaimer of Warranties and Limitation of Liability.

- a. UNLESS OTHERWISE SEPARATELY UNDERTAKEN BY THE LICENSOR, TO THE EXTENT POSSIBLE, THE LICENSOR OFFERS THE LICENSED MATERIAL AS-IS AND AS-AVAILABLE, AND MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND CONCERNING THE LICENSED MATERIAL, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHER. THIS INCLUDES, WITHOUT LIMITATION, WARRANTIES OF TITLE, MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, ABSENCE OF LATENT OR OTHER DEFECTS, ACCURACY, OR THE PRESENCE OR ABSENCE OF ERRORS, WHETHER OR NOT KNOWN OR DISCOVERABLE. WHERE DISCLAIMERS OF WARRANTIES ARE NOT ALLOWED IN FULL OR IN PART, THIS DISCLAIMER MAY NOT APPLY TO YOU.
- b. TO THE EXTENT POSSIBLE, IN NO EVENT WILL THE LICENSOR BE LIABLE TO YOU ON ANY LEGAL THEORY (INCLUDING, WITHOUT LIMITATION, NEGLIGENCE) OR OTHERWISE FOR ANY DIRECT, SPECIAL, INDIRECT, INCIDENTAL, CONSEQUENTIAL, PUNITIVE, EXEMPLARY, OR OTHER LOSSES, COSTS, EXPENSES, OR DAMAGES ARISING OUT OF THIS PUBLIC LICENSE OR USE OF THE LICENSED MATERIAL, EVEN IF THE LICENSOR HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH LOSSES, COSTS, EXPENSES, OR DAMAGES. WHERE A LIMITATION OF LIABILITY IS NOT ALLOWED IN FULL OR IN PART, THIS LIMITATION MAY NOT APPLY TO YOU.
- c. The disclaimer of warranties and limitation of liability provided above shall be interpreted in a manner that, to the extent possible, most closely approximates an absolute disclaimer and waiver of all liability.

Section 6 -- Term and Termination.

- a. This Public License applies for the term of the Copyright and Similar Rights licensed here. However, if You fail to comply with this Public License, then Your rights under this Public License terminate automatically.
- b. Where Your right to use the Licensed Material has terminated under Section 6(a), it reinstates:
 - 1. automatically as of the date the violation is cured, provided it is cured within 30 days of Your discovery of the violation; or
 - 2. upon express reinstatement by the Licensor.

For the avoidance of doubt, this Section 6(b) does not affect any right the Licensor may have to seek remedies for Your violations of this Public License.

- c. For the avoidance of doubt, the Licensor may also offer the Licensed Material under separate terms or conditions or stop distributing the Licensed Material at any time; however, doing so will not terminate this Public License.
- d. Sections 1, 5, 6, 7, and 8 survive termination of this Public License.

Section 7 -- Other Terms and Conditions.

- a. The Licensor shall not be bound by any additional or different terms or conditions communicated by You unless expressly agreed.
- b. Any arrangements, understandings, or agreements regarding the Licensed Material not stated herein are separate from and independent of the terms and conditions of this Public License.

Section 8 -- Interpretation.

- a. For the avoidance of doubt, this Public License does not, and shall not be interpreted to, reduce, limit, restrict, or impose conditions on any use of the Licensed Material that could lawfully be made without permission under this Public License.
- b. To the extent possible, if any provision of this Public License is deemed unenforceable, it shall be automatically reformed to the minimum extent necessary to make it enforceable. If the provision cannot be reformed, it shall be severed from this Public License without affecting the enforceability of the remaining terms and conditions.

- c. No term or condition of this Public License will be waived and no failure to comply consented to unless expressly agreed to by the Licensor.
- d. Nothing in this Public License constitutes or may be interpreted as a limitation upon, or waiver of, any privileges and immunities that apply to the Licensor or You, including from the legal processes of any jurisdiction or authority.

=====
Creative Commons is not a party to its public licenses. Notwithstanding, Creative Commons may elect to apply one of its public licenses to material it publishes and in those instances will be considered the "Licensor." The text of the Creative Commons public licenses is dedicated to the public domain under the CC0 Public Domain Dedication. Except for the limited purpose of indicating that material is shared under a Creative Commons public license or as otherwise permitted by the Creative Commons policies published at creativecommons.org/policies, Creative Commons does not authorize the use of the trademark "Creative Commons" or any other trademark or logo of Creative Commons without its prior written consent including, without limitation, in connection with any unauthorized modifications to any of its public licenses or any other arrangements, understandings, or agreements concerning use of licensed material. For the avoidance of doubt, this paragraph does not form part of the public licenses.

Creative Commons may be contacted at creativecommons.org.

Trademarks

Choupo is a trademark of TalentGround Lda.

Anthropic, Claude, and Claude Code are trademarks or registered trademarks of Anthropic, PBC.

ParaView is a trademark of Kitware Inc.

Contents

Preface — read this first	1
1 Quick start	2
1.1 Permanent shell setup	2
1.2 Build modes	2
2 The four shell helpers	3
3 Anatomy of a case directory	4
3.1 controlDict	4
3.2 flowsheetDict	4
3.3 Units	7
3.4 thermoPackage	8
3.5 Optional dicts	8
4 Built-in unit operations	9
5 Built-in thermodynamic models	10
5.1 Activity models (liquid γ)	10
5.2 Equation of state (vapour φ)	10
5.3 Vapour pressure (per component)	10
5.4 Heat capacity (per component, optional)	10
5.5 Phase types (multi-phase / LLE)	10
6 Sensitivity, optimisation, parameter estimation	11
6.1 Sensitivity sweep — <code>type sweep;</code>	11
6.2 Parameter estimation	11
6.3 Optimisation — <code>type optimization;</code>	12
7 Equipment sizing and costing	13
7.1 Sizing — <code>sizing { units (...) }</code>	13
7.2 Costing — <code>costing { method... }</code>	13
8 Tour of the tutorials	15
9 Output anatomy	16
10 The web GUI: a runner and visualiser	17
10.1 Starting the GUI	17
10.2 The layout	17
10.3 Interactions	17
11 Building your own case	18
12 Troubleshooting	19
13 Where to look next	20

Preface — read this first

Engineers used to text-driven, file-first tools will feel at home. Newcomers will feel curious, then liberated. This is an *educational* process simulator: every Newton iteration, every K -value, every Wegstein step is visible both in the source and at runtime. If something surprises you, the answer is always one **grep** away.

This guide walks you from a cold install to running, modifying, and building your own cases. A dozen-odd short chapters; each opens with a *What you'll learn* note and closes with a hands-on cue. Boxes mark crystal-clear takeaways, common pitfalls, and small experiments you should run as you read.

1 Quick start

What you'll learn. How to build the simulator once, set up your shell environment, and run your first tutorial in under five minutes.

Build once, source the shell environment once, then run any tutorial:

```
cd /path/to/Choupo
make all                      # release build
source etc/bashrc            # adds bin/ + project root to PATH

listCases                    # available tutorials
runCase tutorials/steady/flash/flash01_benzene_toluene # run one
less tutorials/steady/flash/flash01_benzene_toluene/log.choupoSolve
```

Tutorials live under `tutorials/<category>/<name>/`, where the category is one of `steady/` (`choupoSolve`), `batch/` (`choupoBatch`), `ctrl/` (`choupoCtrl`), `props/` (`choupoProps`), or `plant/` (multi-sector cases). `runCase` reads each case's `controlDict.application` and dispatches to the right binary automatically — you never pick the executable by hand. After your first run a symlink (e.g. `./choupoSolve`) points to the matching `build/linux64Gcc/` binary.

1.1 Permanent shell setup

Append this line to `~/.bashrc` so every new terminal has the environment ready:

```
source /path/to/Choupo/etc/bashrc
```

A fresh terminal then greets you with:

```
Choupo environment loaded.
CHOUPO_HOME = /path/to/Choupo
Binaries      : choupoSolve, choupoBatch, choupoCtrl, choupoProps
Commands available: runCase, runApplication,
                  listCases, cleanCase, cdOPE, cdTut
```

1.2 Build modes

```
make all          # default --- release, -O2
make MODE=debug all # -O0 -g, parallel tree under build/linux64GccDebug
make print        # show resolved variables
make clean        # remove current mode
make distclean    # remove the whole build/ tree
```

Both modes coexist; the `choupoSolve` symlink points to the *last* binary built.

Key idea. You never recompile to switch cases. Build once, source once, then all you do is edit text dictionaries and re-run.

Try it now. Run `runCase tutorials/steady/flash/flash01_benzene_toluene` and then `less tutorials/steady/flash/flash01_benzene_toluene/log.choupoSolve`. Notice the per-iteration trace under “Rachford-Rice”: that is the simulator showing its work.

2 The four shell helpers

What you'll learn. Five short commands cover every interaction you'll have with the simulator from the shell.

After sourcing `etc/bashrc` you have:

Command	Purpose
<code>choupoSolve <case></code>	Run a case; write everything to stdout.
<code>runCase [<case>]</code>	Run + redirect to <code>log.choupoSolve</code> in the case dir. Defaults to current dir.
<code>runApplication choupoSolve <args></code>	redirect to <code>log.<appName></code> in cwd.
<code>listCases</code>	Tabulate all tutorials with one-line descriptions.
<code>cleanCase [<case>]</code>	Remove <code>log.*</code> and <code>*.log</code> from a case (never touches source).
<code>cdOPE</code>	<code>cd</code> to the project root.
<code>cdTut</code>	<code>cd</code> to <code>tutorials/</code> .

Two equivalent workflows:

```
# From anywhere (use $CHOUPPO_HOME):
runCase $CHOUPPO_HOME/tutorials/steady/flash/flash01_benzene_toluene

# from inside the case dir:
cd tutorials/steady/flash/flash01_benzene_toluene
runCase                                # writes log.choupoSolve here
```

3 Anatomy of a case directory

What you'll learn. Every case is a folder of plain-text dicts. Two are required; three are optional. Nothing else.

```
caseName/
|-- system/
|   |-- controlDict          required  --- verbosity, [future] time control
|   |-- flowsheetDict        required  --- topology: streams + units + connections
|   |-- solverDict            optional  --- per-unit-op solver options (overrides)
|   |-- outerDict             optional  --- sensitivity / optimisation / param-est
|   '-- postDict              optional  --- post-processing chain (sizing, costing)
|-- constant/
|   |-- thermoPackage         required  --- components + gamma-phi models / phases
|   '-- reactions              optional  --- named-reaction library (CSTR / PFR)
'-- log.choupoSolve           produced  --- written by runCase
```

Key idea. A standalone unit operation is just a `flowsheetDict` of length 1. There is no special “single-unit” mode — one consistent case format for everything.

3.1 controlDict

The simplest valid file:

```
verbosity 3; // 0 silent / 1 errors / 2 warnings / 3 info / 4 debug
```

For pedagogical use you want `verbosity 3` — that prints every Newton iteration, every Wegstein step, every K -value evaluation. Set lower only when sweeping the simulator thousands of times.

3.2 flowsheetDict

Streams + units + (optional) tear-stream declaration for recycles.

```
streams
{
  feed
  {
    F      100.0 kmol/h; // industrial unit; parser
    T      370.0 K;     // converts to SI canonical
    P      1.0 bar;     // (kmol/s, K, Pa) on the fly
    composition { benzene 0.40; toluene 0.60; }
  }
}

units
(
  {
    name      flash01;
    type      isothermalFlash;
    in        feed;
    outputs    (liquid vapor); // names assigned to producedStreams[0..k]

    operation { T 370.0 K; P 1.0 bar; }
  }
);
```

Units on every scalar. Since, any scalar in a dict may carry a unit suffix. The parser converts to canonical SI internally (Pa, kmol/s, K, m, m², m³, J, kg) and tags the entry with its physical dimensions, so a typo (P 1 kg;) is caught with a crystal-clear error before the solver runs. See §3.3 for the three accepted forms and the `controlDict.units {...}` block.

Composition basis. Five ways to declare a stream's flow and composition — pick exactly one:

```
// (a) F + molar fractions
F          100.0 kmol/h;
molarComposition { benzene 0.50; toluene 0.50; }

// (b) F + mass fractions
F          5000 kg/h;
massComposition { benzene 0.40; toluene 0.60; }

// (c) legacy alias of (a)
F          100.0 kmol/h;
composition { benzene 0.50; toluene 0.50; }

// (d) per-species molar flows -- F and x_i derived
molarFlows
{
    benzene  40.0 kmol/h;
    toluene  60.0 kmol/h;
}

// (e) per-species mass flows -- F and x_i derived
massFlows
{
    benzene  3124.4 kg/h;
    toluene  5528.4 kg/h;
}
```

Forms (d) and (e) are the natural way to describe a feed in process design (“mix 40 kmol/h of A with 60 kmol/h of B”); F total and composition are computed from the sum. Forms (a) / (b) / (c) are the textbook way (total flow + composition).

Internally the simulator always stores mole fractions, so mass-basis inputs (b, e) are converted at parse time via $x_i = (w_i/MW_i) / \sum_j w_j/MW_j$.

Sum check. A composition block is always renormalised to $\sum_i x_i = 1$, but the parser also checks the user-supplied sum and either passes silently or shouts, depending on how far it drifts from 1. Four regimes:

Input	Behaviour	Why
$\sum_i = 1.0$ exact	silent — values used as data	textbook input
$ \sum_i - 1 \leq 10^{-3}$	silent — normalised	rounding drift
$ \sum_i - 1 > 10^{-3}$	warning + normalised	likely typo
$\sum_i = 0$	fatal error — simulator stops	no feed at all

The warning emitted on the third case names the stream, the block, and the actual sum:

```
WARNING: Stream 'feed': molarComposition sums to 0.9 (should be 1.0);
        values will be normalised
```

Why renormalise instead of rejecting outright? Two pedagogical conveniences:

- Shortcut for equimolar feeds: { A 1; B 1; } is an unambiguous 50/50 without having to type the arithmetic.
- Tolerance for rounding: { A 0.333; B 0.333; C 0.333; } (sum 0.999) should not abort a simulation.

The threshold 10^{-3} is tight enough to catch a typo like { A 0.4; B 0.5; } (forgot to write 0.6) yet loose enough to wave through legitimate rounding.

Flow-based forms are not normalised. When the stream is declared via `molarFlows` or `massFlows` (forms d / e above), the absolute species flows are the source of truth and the parser sums them to find F_{total} . There is no “should sum to 1” check; the only fatal case is $F_{\text{total}} = 0$ (an empty feed).

Sixth form: state keyword + flash on import. The five flavours above all assume you know T and P ahead of time. But many streams in a problem have an *under-specified* thermodynamic state that the simulator should resolve on import. Two examples that come up immediately:

- Heating utility (live steam at 170 kPa): the engineer knows the header pressure; the temperature is whatever T_{sat} corresponds to it; the flow is whatever the consuming unit demands.
- Cooling water (saturated liquid at supply pressure): same logic, on the liquid side of the saturation curve.

For these streams, the user declares a **state** keyword and only the variables that are physically known. The parser completes the rest by inverting the dominant component’s saturation curve:

```
liveSteam
{
  P          170.0 kPa;
  state      saturatedVapour;    // -> T computed from
                                //      Psat-1(170 kPa)

  molarComposition { water 1.0; }
  // no F! The consuming unit op (the evaporator's heating
  // chest) computes the actual steam demand and the simulator
  // writes it back into the stream registry after solve().
  // The Streams panel then shows the real consumption.
}

coolingWater
{
  T          298.15 K;
  state      saturatedLiquid;    // -> P computed from
                                //      Psat(298.15) for water

  molarComposition { water 1.0; }
  F          50.0 kmol/h;        // here F is known a priori
                                //      (plant-side metered flow)
}
```

Supported **state** values:

state	Computes	Sets vf
saturatedVapour	T from P , or P from T	1.0
saturatedLiquid	T from P , or P from T	0.0
subcooledLiquid	— (validates $T < T_{\text{sat}}(P)$)	0.0
superheatedVapour	— (validates $T > T_{\text{sat}}(P)$)	1.0

Restrictions:

- Currently **state** works for *pure-component* streams (one species with $x_i \geq 0.999$). Multi-component saturated states (bubble-point / dew-point of a mixture) are scheduled.
- If you declare *both* T and P alongside a saturated state, the parser checks consistency against $P_{\text{sat}}(T)$ and throws if they disagree by more than 5%.
- Omitting F is only allowed for streams declared as **saturatedVapour** or **saturatedLiquid** — the utility pattern. Subcooled / superheated streams require an explicit F since they are not demand-driven utilities.

Consistency check on input streams (the bigger picture). The **state** keyword is one piece of the *flash-on-import* contract: every input stream becomes a single point in thermodynamic-state space, fully specified, before any unit op runs. Internally the simulator already does this for ALL streams — whether or not the user wrote **state**. Three checks happen at parse time:

1. Composition normalisation (the four regimes documented in the Sum-check subsection).
2. Phase-state consistency (the **state**-keyword paths above). Without an explicit **state**, a stream with T , P , $\mathbf{z}$ given is taken as “whatever the flash regime turns out to be”; multi-component streams left under-specified will go through a Rachford-Rice flash on first use by a unit op.
3. Dimension cross-check on every scalar value (any unit suffix on T , P , F etc. is validated against the expected physical dimensions; see §3.3).

For recycles, declare the cycle-breaking streams and the Wegstein parameters:

```

tearStreams      (recycle );
recycleTol        1e-5;
recycleMaxIter    60;
recycleWegsteinQmin -0.5;
recycleWegsteinQmax 0.0;

```

3.3 Units

Choupo uses canonical SI internally: Pa for pressure, kmol/s for molar flow, K for temperature, m for length, m² for area, m³ for volume, J for energy, kg for mass, kmol for amount. You almost never want to type those raw, so any scalar value in a dict may carry a unit suffix. The parser converts the value to SI on the fly and remembers what dimension it was — a typo in the unit is caught with a clear error before the solver runs.

Three accepted forms.

```

P      1      bar;           // (a) named-unit shortcut
A_w    [-1  2  1  0  0] 1.5e-12; // (b) bracket form
                                     //      [M L T Theta N] + SI value
R      0.5;                 // (c) raw SI (no declaration)

```

Form (a) is what you will type 95% of the time. Form (b) is the escape hatch for quantities with no short named unit (membrane permeabilities, exotic transport coefficients). Form (c) says “trust me, the number is in canonical SI” — the parser cannot check anything, so a typo here passes silently.

Named units shipped by Choupo. A non-exhaustive overview of the most useful aliases (see `src/core/Units.cpp` for the full table):

Quantity	Canonical SI	Named-unit aliases
pressure	Pa	Pa, kPa, MPa, bar, atm, psi, mmHg, torr
molar flow	kmol/s	kmol/s, kmol/h, mol/s, mol/h
mass flow	kg/s	kg/s, kg/h, g/s, g/h, t/h
volumetric flow	m ³ /s	m3/s, m3/h, L/s, L/h, L/min
temperature	K	K, degC, C, Celsius, degF, F, Fahrenheit
length	m	m, mm, cm, km, in, inch, ft
area	m ²	m2, m^2, cm2, mm2
volume	m ³	m3, m^3, L, mL
time	s	s, sec, min, h, hr, day
mass	kg	kg, g, t, ton, tonne
energy	J	J, kJ, MJ
power	W	W, kW, MW, power
concentration	kmol/m ³	kmol/m3, mol/m3, mol/L, M
density	kg/m ³	kg/m3, g/cm3, g/mL
viscosity	Pa·s	Pa.s, cP
membrane A_w	m/(s·Pa)	m/s/Pa, m/s/bar

What the student sees on a bad unit. Asking for a pressure but typing `P 1 kg`; produces:

```

ERROR: Dictionary 'feed' ():
  parameter 'P' expected dimensions [1 -1 -2 0 0] (Pa)
  but the dict declared [1 0 0 0 0] (kg). Check the unit
  suffix on this entry.

```

The same check guards every parameter that the unit operations read (`T`, `F`, `V_R`, `area`, `length`, `k_film`, `P_permeate`, ...).

Output preferences. By default Choupo prints flows in kmol/h, pressures in bar, temperatures in K, sizes in m / m² / m³ — the chem-eng convention. To override, drop a `units` block in `controlDict`:


```

units
{
    pressure      atm;          // default bar
    flow          kmol/s;       // default kmol/h
    temperature    K;           // default K
    length         m;
    volume         L;
    energy         kJ;
    power          kW;
}

```

The simulator internals stay in SI; only the printed output changes. Mismatched dimensions in the override (e.g. `pressure m`;) are reported and ignored.

Decimal precision. An optional `precision` block in `controlDict` tunes the number of decimal digits per quantity category:

```

precision
{
    pressure      3;           // default 3
    temperature    2;           // default 2
    flow          4;           // shorthand for molarFlow + massFlow
    composition    4;           // mole / mass fractions
}

```

Other accepted keys: `molarFlow`, `massFlow`, `volumetricFlow`, `length`, `area`, `volume`, `time`, `mass`, `amount`, `energy`, `power`, `concentration`, `density`, `velocity`, `viscosity`, `diffusivity`. Omit any key to keep its default. The defaults match the hardcoded formatting so existing tutorials look identical.

3.4 thermoPackage

Two equivalent syntaxes — legacy (VLE only) and explicit (general, supports LL / VLL):

```

// Legacy:
components      (benzene toluene );
activityModel    { model ideal; }
equationOfState { model idealGas; }

// Explicit:
components (water nButanol );
phases
(
    { name aqueous; type liquid; activity { model NRTL; pairs (... ); } }
    { name organic; type liquid; activity { model NRTL; pairs (... ); } }
);

```

3.5 Optional dicts

- `solverDict` — defaults per unit-op type, overridable by each unit. E.g. `isothermalFlash { tolerance 1e-8; outerAccelerator wegstein; }`.
- `outerDict` — see §6.
- `postDict` — see §7.
- `constant/reactions` — named-reaction library. CSTR / PFR reference reactions by name: `reaction ethanol_to_water_test;`.

4 Built-in unit operations

What you'll learn. The unit-op types available (around two dozen, plus aliases such as `flash`, `HX`, `column`, `FUG`, `membraneSW`) and what each operation block must contain. The table below lists the workhorses; `UnitOperation::availableTypes()` prints the full registry at runtime.

type	What it does	Required keys
<code>isothermalFlash</code> (<i>alias</i> <code>flash</code>)	Multi-component V–L or L–L flash	T, P, phaseSet VL/LL
<code>adiabaticFlash</code>	Adiabatic two-phase flash; solves T_{out}	P
<code>bubbleT</code>	Bubble temperature at given $\mathbf{x}$, P	P
<code>dewT</code>	Dew temperature at given $\mathbf{y}$, P	P
<code>cstr</code>	Single-reaction liquid CSTR	V_R, reaction <name>
<code>pfr</code>	Plug-flow reactor (RK4 over volume)	V_R, nSteps, reaction <name>
<code>heatExchanger</code> (<i>alias</i> <code>HX</code>)	Liquid heater / cooler	one of Tout or Q
<code>mixer</code>	$N \rightarrow 1$ mixer with energy balance	uses inputs (a b c)
<code>splitter</code>	$1 \rightarrow N$ tee with composition split	fractions (f1 f2 ...)
<code>distillationColumn</code> (<i>alias</i> <code>column</code>)	Multi-stage column (Wang-Henke or simultaneous MESH)	nStages, feedStage, refluxRatio, distillateRate, feedQuality, P
<code>shortcutColumn</code> (<i>alias</i> <code>FUG</code>)	Fenske-Underwood-Gilliland shortcut column	lightKey, heavyKey, refluxRatio, P
<code>absorber / stripper</code>	Multi-stage gas absorber / stripper	nStages, P
<code>gibbsReactor</code>	Equilibrium reactor by Gibbs-energy minimisation	T or adiabatic; reactions (...)
<code>compressor / turbine / pump</code>	Rotating equipment; spec $W_{\text{shaft}} + \eta$ (P_{out} is a result)	W_{shaft} , η
<code>electricLoad</code>	Sink for shaft work via an energy wire	uses energyInputs
<code>valve</code>	Isenthalpic throttle to a target pressure	P_{out}
<code>evaporator</code>	Single-effect evaporator with heating duty	P (+ heat)
<code>sprayDryer / solidDryer</code>	Gas–solid drying (constant + falling rate)	T_{gas} , sample drying curve
<code>crystalliser</code>	Population-balance crystalliser	nucleation/growth kinetics
<code>cyclone / bagFilter / gasSolidSplitter</code>	Gas–solid separation (5 cyclone sub-models)	geometry per model
<code>spiralWoundModule</code> (<i>alias</i> <code>membraneSW</code>)	NF/RO spiral-wound element (solution-diffusion)	membrane name, area, P_{permeate} , k_{film}

Each unit also accepts (top-level, not inside `operation`) overrides for solver tolerances — same keys as in `solverDict`.

Energy ports and heat integration. Rotating equipment, heaters, and distillation reboilers/condensers exchange *work* and *heat* through typed energy ports (`energyOutputs` / `energyInputs`) rather than process streams. A turbine's shaft work can drive a compressor or an `electricLoad`; a column's reboiler heat can preheat another unit (a forward heat-link). This layer — the W/Q wires, the `column.condenser` / `column.reboiler` port names, the auto-allocation-by-temperature rule, and the `utilityAllocation` report — is documented in `docs/ai/energy.md`.

5 Built-in thermodynamic models

What you'll learn. The four families of models that together cover everything the simulator does: activity coefficients, equation of state, vapour pressure, and heat capacity.

5.1 Activity models (liquid γ)

model	Notes
ideal	$\gamma_i = 1$ — the Raoult limit.
NRTL	Renon-Prausnitz; needs binary pairs.
Wilson	Wilson 1964; needs binary pairs + V_{liq} per component.

NRTL pair entry:

```

pairs
(
  {
    i      ethanol;      j      water;
    a_ij   -0.8009;      b_ij   246.18;      // tau_ij = a_ij + b_ij/T
    a_ji   3.4578;      b_ji   -586.0809;
    alpha  0.30;
  }
);

```

5.2 Equation of state (vapour φ)

model	Notes
idealGas	$\varphi_i = 1$; $PV = RT$. Adequate for the low-pressure tutorials.

5.3 Vapour pressure (per component)

Stored in `data/standards/components/<name>.dat`:

model	Form
Antoine	$\log_{10} P^{\text{sat}}[\text{bar}] = A - B/(T[\text{K}] + C)$

5.4 Heat capacity (per component, optional)

model	Form
polynomial	$C_p = a_0 + a_1T + a_2T^2 + a_3T^3 + a_4T^4$, [J/(mol·K)]

5.5 Phase types (multi-phase / LLE)

type	Notes
vapor	wraps an EoS
liquid	wraps an ActivityModel + uses Antoine for P^{sat}
solid	stub: declared interface for future SLE / inert solids

6 Sensitivity, optimisation, parameter estimation

What you'll learn. How to wrap the simulator in an outer loop without writing a single line of C++.

Drop a `system/outerDict` into a case and the simulator is wrapped in an outer loop.

6.1 Sensitivity sweep — type sweep;

Vary one design value across a grid and harvest responses:

```
type          sweep;

parameter
{
    target      units[0].operation.refluxRatio;    // JSON-path-like syntax
    range       ( 1.5  5.0 );
    nPoints     10;
}

responses
(column01.T_top
 column01.T_bottom
 column01.x_D_LK
 column01.V_strip
);

report
{
    format      csv;
    file        sweep_results.csv;
}
```

Each response is `unitName.kpiName` (a unit-op KPI) or `streamName.field` where field is F, T, P, or vf.

Try it now. `runCase tutorials/steady/optimisation/sensitivity01_column_reflux` produces `sweep_results.csv`. Plot column 6 against column 2 with any tool; you'll see the classic " V_{strip} asymptote" as the reflux ratio grows past the minimum.

6.2 Parameter estimation

A dedicated concrete driver, `type fitBinaryPair;`, regresses the four interaction parameters of one binary pair against experimental bubble-temperature data using hand-rolled Levenberg-Marquardt (Marquardt diagonal damping, central finite-difference Jacobian, 3-up/5-down trust-region update of λ):

```
type          fitBinaryPair;

pair          ethanol-water;
data          constant/experiments/ethanol-water-101kPa;
unit          bt01;          % the bubbleT unit that emits T_bubble

% Optional (defaults shown):
% maxIter     30;           tolerance 1.0e-6;
% fdStep      1.0e-4; lambda0    1.0e-3;
```

The dataset is a dict with a flat two-column list:

```
P      1.01325;
data
(
    0.05  361.95
    0.10  359.95
    ...
    0.95  351.45
);
```

The driver writes a dated proposal to `constant/binaryPairs/<model>/<c1>-<c2>.fit-<YYYY-MM-DD>.dat`; the human promotes it manually with `mv` once inspected. Worked example: `tutorials/fitNRTL01_ethanol_water/`, with detailed walk-through in its `README.md`.

6.3 Optimisation — type optimization;

`type optimization`; drives a hand-rolled Nelder-Mead simplex to minimise a target KPI (e.g. annualised cost) over one or more decision variables, re-running the whole simulator — and, if a `postDict` is present, the sizing + costing chain — at every evaluation. Two worked examples ship: `tutorials/steady/optimisation/optim01_column_reflux` (minimises `column01.V_strip` over reflux ratio + feed stage) and `tutorials/steady/optimisation/optim02_process_cost` (minimises the total Guthrie module cost over reactor volume + heater outlet temperature).

7 Equipment sizing and costing

What you'll learn. How a single `postDict` turns a converged flowsheet into a vessel list, wall thicknesses, weights, and Guthrie/Turton bare-module costs in EUR.

Drop a `system/postDict` and the simulator result is post-processed.

7.1 Sizing — `sizing { units (...) }`

```
sizing
{
  units
  (
    {
      unitName    reactor;
      type        stirredTank;
      material     SS316;
      designRules
      {
        L_over_D      2.5;
        pressureDesign 3.0;
        corrosionAllow 0.003;      // m
        jointEfficiency 1.0;
      }
    }

    {
      unitName    heater;
      type        shellTubeHX;
      material     SS304;
      designRules
      {
        U            600.0;      // W/(m^2 K)
        LMTD         10.0;      // K
        pressureDesign 2.0;
      }
    }
  );
}
```

`type` selects an `EquipmentSize` model. At: `stirredTank` (vessels including CSTRs and reactor surge drums) and `shellTubeHX`. Materials come from `data/standards/materials/*.dat` — built-in: `carbonSteel`, `SS304`, `SS316`, `aluminium`.

7.2 Costing — `costing { method... }`

```
costing
{
  method      Guthrie;
  year        2026;
  cepci       820;
  cepci2001   397;
  usdToEur    0.92;
}
```

The `Guthrie` model uses Turton-style purchased-cost correlations, ASME wall-thickness pressure factor for vessels, polynomial F_P for heat exchangers, then $C_{BM} = C_p(B_1 + B_2 F_M F_P)$, and $C_{TM} = 1.18 C_{BM}$.

The post-processor prints a per-unit cost table to stdout:

```
===== Equipment Costing =====
Method: Guthrie   Year: 2026.0   CEPCI: 820.0
```

unit	equipment	F_M	F_P	C_purchased	C_bare_mod	C_total_mod
heater	shellTubeHX	2.34	1.09	365 249	2 146 753	2 533 169
reactor	stirredTank	3.05	1.00	2 055	16 033	18 918
TOTALS (EUR)				367 304	2 162 786	2 552 087

8 Tour of the tutorials

What you'll learn. Choupo ships 131 tutorials under `tutorials/{steady,batch,ctrl,props,plant}/`; `listCases` tabulates them all with one-line descriptions. The selection below is the steady-state on-ramp — pick the one closest to what you want to do and edit from there.

Tutorial	What you learn
<code>flash01_benzene_toluene</code>	First run: Raoult VLE, Rachford-Rice, Newton iteration visible.
<code>flash02_ethanol_water</code>	NRTL + Wegstein outer loop (~ 7 iter vs ~ 29 direct subst.).
<code>flash03_wilson_ethanol_water</code>	Same problem with Wilson — model uncertainty visible.
<code>bubbleT01_ethanol_water</code>	Find T such that $\sum K_i x_i = 1$.
<code>dewT01_ethanol_water</code>	Find T such that $\sum y_i / K_i = 1$.
<code>adiabaticFlash01_benzene_toluene</code>	$Q = 0$ — solve for T_{out} ; uses C_p and ΔH_{vap} (Watson).
<code>cstr01_first_order</code>	Newton on extent ξ ; Damköhler & analytic check.
<code>pfr01_first_order</code>	RK4 by hand; reproduces $X = 1 - e^{-k\tau}$.
<code>hx01_heater</code>	Specify T_{out} or Q , compute the other; Newton 1-D.
<code>process01_reactor_flash</code>	First flowsheet: CSTR $\rightarrow$ flash; mass-balance closure.
<code>process02_reactor_hx_flash</code>	Three units; T propagates through the chain.
<code>process02_with_design</code>	The above + sizing + Guthrie costing — full CAPEX in one run.
<code>process03_recycle</code>	Reactor-separator-recycle: tear stream + Wegstein outer loop.
<code>column01_benzene_toluene</code>	Multi-stage column, Wang-Henke bubble-point.
<code>sensitivity01_column_reflux</code>	Sweep R over $[1.5, 5]$ to CSV with V_{reboiler} vs purity.
<code>optim01_column_reflux</code>	Nelder-Mead: minimise V_{strip} over reflux + feed stage.
<code>optim02_process_cost</code>	Nelder-Mead on total Guthrie module cost (sizing + costing per evaluation).
<code>process04_research_workflow</code>	Case-local <code>constant/binaryPairs/<model>/<pair>.dat</code> instead of inline pairs — the lookup-order pattern.
<code>fitNRTL01_ethanol_water</code>	<code>fitBinaryPair</code> outer driver: LM regression of NRTL ethanol/water against 1 atm bubble- T data (see §6.2).
<code>extraction01_waterButanol</code>	LL extraction with NRTL — <i>experimental</i> (see §12).

9 Output anatomy

What you'll learn. What every section of a run's log means, and why the inner iteration trace is the part you should read.

A typical run with `verbosity 3` prints, in order:

1. **Banner** — version + licence.
2. **Case header** — case dir, database root, optional-dicts presence.
3. **Components and phases** declared.
4. **Topology** of the flowsheet — source streams + the per-unit blocks.
5. **Per-unit run** — for each unit operation:
 - Inputs echo ($F, T, P, \mathbf{z}$).
 - Inner iteration trace (Newton steps with $g(x)$, dg/dx , ΔV).
 - For γ -models: outer Wegstein loop with γ values per iter.
 - Result block (regime, V/F , $\mathbf{x}$, $\mathbf{y}$, $\mathbf{K}$).
6. **Flowsheet summary** — final stream table.
7. **Outer driver / post-processor output** (if applicable).

Key idea. The inner iteration trace is where the simulator stops being a black box. Convergence ratios in the trace tell you whether Newton is converging quadratically (good) or linearly (problem is ill-conditioned).

10 The web GUI: a runner and visualiser

What you'll learn. How to open a case in the browser, run it, and read the result — and why the GUI never edits your dicts.

Choupo ships a browser front-end under `gui/`. Its design stance is deliberate and worth stating up front: **the GUI is a runner and visualiser, not an editor**. Cases are authored as plain-text dicts under `tutorials/<name>/system/` and `constant/`, edited with any text editor (increasingly with LLM help). The GUI loads those dicts, draws the flowsheet, runs the solver *in the browser*, and renders the log + plots. The dicts on disk remain the single source of truth; per-case scalar tweaks in the GUI are downloaded back to disk, never silently mutated. The architectural inspiration is ParaView (a DSL plus a viewer), not a block-diagram GUI editor.

10.1 Starting the GUI

```
make wasm-gui      # build the WASM solver (choupoSolve + choupoProps)
bin/runGui         # boots the dev server and opens a browser window
bin/runGui --kill  # stops everything
```

10.2 The layout

The shell is a thin top bar over a full-screen flowsheet canvas — no left tree, no docked right panel, no bottom tab strip. Auxiliary views (log, plots, stream table) open on demand as workspaces. Selecting a unit or stream raises a floating selection card in the top-right corner; the card is read-only (labels, units, help tooltips driven by the operation schemas).

10.3 Interactions

- **Single-click** a node or edge — raises the floating selection card.
- **Double-click** — pops the unit out (or drills into a composite sector).
- **Drag** nodes, edge bends, and connection points to lay the flowsheet out; the layout persists per case in the browser's `localStorage`.
- **Stream-class toggle** — show/hide whole classes of streams (e.g. energy wires) to declutter the canvas.
- **Run** — streams the same log you would see from `runCase`; **Stop** cancels mid-solve.

Stream colour encodes phase and comes straight from the solver (`vf` + solid mass fraction); it never changes on hover or selection — selection is signalled by a halo/ring, not a recolour.

Common pitfall. MOCK vs WASM. If the WASM solver has not been built (`make wasm-gui`), the GUI silently falls back to a *mock* adapter that returns canned numbers so the UI still renders. The Run badge reads **MOCK** in that case, not **WASM** — check it before trusting any result. After changing a unit op or any C++ source, rebuild the WASM (it is a separate artefact from the native binary) or the browser keeps running stale code.

11 Building your own case

What you'll learn. The fastest path from a blank idea to a converged result is to clone the tutorial closest to your idea and edit.

```
cp -r tutorials/steady/flash/flash01_benzene_toluene tutorials/steady/myFlash
$EDITOR tutorials/steady/myFlash/system/flowsheetDict
$EDITOR tutorials/steady/myFlash/constant/thermoPackage
runCase tutorials/steady/myFlash
```

Adding a missing component. Two locations are available — pick the right tier for what you are adding:

- `data/standards/components/<name>.dat` — contribution to the *shared catalogue* of well-curated chemicals.
 - `<case>/constant/components/<name>.dat` — *sample-specific or one-off data*: a particular powder's sorption isotherm, a custom membrane solute, a laboratory measurement. Per-case files are overlaid field-by-field on top of the standard catalogue, so a slim file with just the bespoke fields is enough — the rest is inherited from the standard entry of the same name.
1. Create the file in the chosen tier, following the same format as the existing standard entries (see Developer Guide § Adding components for the field list). When the file carries a `gibbsFormation` block, declare the **phase** of the tabulated Δh_f° explicitly (`gas` / `liquid` / `solid`) — the value tells the solver which rung of the enthalpy ladder the data is pinned to; see Theory Guide § “Which rung does the data file pin?”.
 2. Add `<name>` to the `components (...)` list in your `thermoPackage`.

Using a plant utility. Heating and cooling services are catalogued separately from process components, under `data/standards/utilities/`. Eight services ship by default — LP / MP / HP steam, cooling water, chilled water, Dowtherm A, HITEC molten salt, and a propylene-glycol refrigeration brine — each with operating conditions and a `cost` in EUR/GJ.

A stream block pulls the catalogue defaults with a single keyword:

```
streams
{
    PlantSteam { utility steamLP; F 60 kmol/h; }
}
```

The parser substitutes `components`, `state`, `P` and `T` from the `steamLP.dat` entry (the user keeps full control — writing `P 3 bar`; alongside the `utility` key overrides the catalogue default for that one case), and marks the stream's internal category to `"steamLP"`. When the case runs, the `utilities` report aggregates every utility-tagged stream by category and emits `reports/utilities/consumption.csv` with the kg/h, MW, and EUR/h delivered by each header. Multiple streams sharing a catalogue name (two heaters tapping the same plant header) sum automatically.

Adding a missing material. Same procedure under `data/standards/materials/`.

Using a reaction in a CSTR / PFR. Declare it in `constant/reactions` and reference it by name in the unit's `reaction <name>;` entry.

12 Troubleshooting

What you'll learn. The seven failure modes that account for nearly every support ticket, and exactly what to do about each.

“Component does not exist” / “Material does not exist”

```
ERROR: Database: component file not found: data/standards/components/foo.dat
```

The component (or material) name is misspelt, or the file is missing. Filenames are case-sensitive. Check `data/standards/components/`.

Build fails after pulling new code

```
make distclean
make all
```

Removes the entire `build/` tree. Necessary when headers change in incompatible ways.

Simulator runs but flowsheet returns NaN

Most common: feed composition does not sum to 1, or a unit-op is fed with an empty stream. Re-read the input echo and verify $F, T, P, \mathbf{z}$ values.

Newton fails to converge

Two situations:

- **Outer Wegstein** (γ -model loop): `qmin` too aggressive. Try `outerAccelerator directSubstitution` for non-ideal LL or azeotropic systems.
- **Inner Newton** (Rachford-Rice or distillation bubble-T): increase `maxIter` and `maxOuterIter`. Near the phase boundary, tighten `compositionTol`.

LL extraction tutorial does not give a physical split

Known issue (still open at) — LL flash converges to the trivial $K = 1$ fixed point in systems with the same γ -model on both phases. Resolution requires second-order Michelsen TPD with Newton — scheduled. The tutorial is shipped as *experimental* and documents this gap.

Sweep run fails on some points

Almost always: the simulator's `maxOuterIter` is too small for the edge points (high reflux ratio, high pressure, ...). Increase `maxOuterIter` in the corresponding unit-operation block in `flowsheetDict`.

Common pitfall. A run that prints $V/F = \text{nan}$ mid-flash almost always means $\log P_i^{\text{sat}}$ was evaluated outside Antoine's valid range (e.g. T too low so Antoine returns a negative pressure), or that a stream upstream had $F = 0$ and emptied the composition vector.

13 Where to look next

- `CLAUDE.md` — conventions, architecture overview, AI-assistant guide.
- `docs/developerGuide.pdf` — adding unit operations, thermo models, components, materials, outer drivers, post-processors.
- `tutorials/<name>/system/*` — every tutorial is a fully-worked example.
- `src/applications/choupoSolve/main.cpp` — the entry point.

Key idea. If a case behaves unexpectedly, *read the log*. Every Newton step, every Wegstein update, every γ value is printed. Black-box debugging is exactly the failure mode Choupo refuses to participate in.

Colophon. This manual was typeset in Latin Modern with the Choupo shared LaTeX preamble. Sources live in `docs/`; rebuild with `cd docs && make`.